Università degli Studi di Palermo

dipartimento di ingegneria unipa

# Introduzione alle stringhe di caratteri

CALCOLATORI ELETTRONICI – FONDAMENTI DI PROGRAMMAZIONE

a.a. 2025/2026

Prof. Roberto Pirrone

CHILAB

# Dichiarazione di una stringa

- La maniera più semplice di dichiarare una stringa è come *vettore di caratteri*

```
char string_var[30];
char str[20] = "Initial value";
```

  - Viene allocato lo spazio in memoria per tanti `char` quanto è il valore tra `[` e `]`

  - Come faccio a gestire stringhe di *lunghezza variabile*?
  - Vengono stampati o caricati *per forza tutti i caratteri*?

# Cosa c’è in una stringa?

- La stringa inizializzata viene terminata con il *carattere nullo* che si rappresenta come `‘\0’` è ha codice ASCII pari a 0

```
char str[20] = "Initial value";
```

| [0] | | | | [4] | | | | [9] | | | | | | [14] | | | | | [19] |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| I | n | i | t | i | a | l | | v | a | l | u | e | \0 | ? | ? | ? | ? | ? | ? |

- I caratteri successivi a `‘\0’` vengono ignorati e non vengono stampati

`char str[7]=”” ;` Cosa c’è nella stringa?

# Cosa c’è in una stringa?

- E se voglio gestire un elenco di nomi?

- Esempio: il calendario dei mesi dell’anno

```
char month[12][10] = {"January", "February", "March", "April",
                      "May", "June", "July", "August",
                      "September", "October", "November",
                      "December"};
```

- Quanto vale `month[3]` ? → `''April''`

- Quanto vale `month[4][2]` ? → `'y'`

# Array di stringhe

- `month` è un ***vettore di stringhe*** !!!

```
char month[12][10]
```

12 … stringhe da 10 caratteri

# Array di stringhe

- `month` è un ***vettore di stringhe*** !!!

`month[3]` …

Seleziona uno tra i 12 e restituisce una *stringa da al più 10 caratteri*

# Array di stringhe

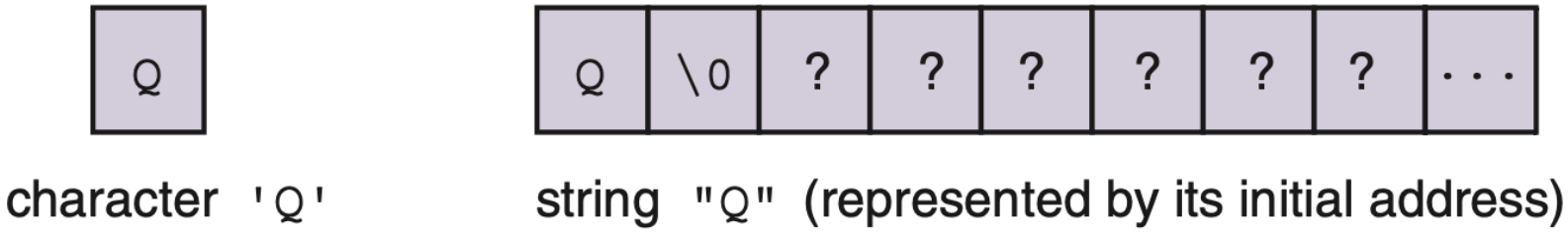
- `month` è un ***vettore di stringhe*** !!!

`month[4][2]` …

Seleziona uno tra i 12 ed uno tra i 10 e restituisce un *carattere*

# Caratteri e stringhe

- Che differenza c'è tra il carattere `'Q'` e la stringa `''Q''` ?




- Lettura di un carattere

```
scanf("%c", &ch)          ch = getchar()
```

# ctype.h

**TABLE 8.3** Character Classification and Conversion Facilities in ctype Library

| Facility | Checks | Example |
|---|---|---|
| `isalpha` | if argument is a letter of the alphabet | `if (isalpha(ch))`<br>`  printf("%c is a letter\n", ch);` |
| `isdigit` | if argument is one of the ten decimal digits | `dec_digit = isdigit(ch);` |
| `islower`<br>`(isupper)` | if argument is a lowercase (or uppercase) letter of the alphabet | `if (islower(fst_let)) {`<br>`  printf("\nError: sentence ");`<br>`  printf("should begin with a ");`<br>`  printf("capital letter.\n");`<br>`}` |
| `ispunct` | if argument is a punctuation character, that is, a noncontrol character that is not a space, a letter of the alphabet, or a digit | `if (ispunct(ch))`<br>`  printf("Punctuation mark: %c\n",`<br>`  ch);` |
| `isspace` | if argument is a whitespace character such as a space, a newline, or a tab | `c = getchar();`<br>`while (isspace(c) && c != EOF)`<br>`  c = getchar();` |

| Facility | Converts | Example |
|---|---|---|
| `tolower`<br>`(toupper)` | its lowercase (or uppercase) letter argument to the uppercase (or lowercase) equivalent and returns this equivalent as the value of the call | `if (islower(ch))`<br>`  printf("Capital %c = %c\n",`<br>`  ch, toupper(ch));` |

# Lettura e scrittura delle stringhe

- Comportamento con `scanf`
- *Non è* necessario il carattere `&`
  - In quanto vettore di caratteri, il nome della stringa *è già un riferimento ad una locazione di memoria*

```
char str[5]="";
printf("> ");
scanf("%9s",str);
printf("%s\n",str);
```

Che succede?

# Lettura e scrittura delle stringhe

- Comportamento con `scanf`
- *Non è* necessario il carattere `&`
  - In quanto vettore di caratteri, il nome della stringa *è già un riferimento ad una locazione di memoria*

```
char str[7]="";
char str1[7]="";

printf("> ");
scanf("%s,%s",str,str1);
printf("%s\n%s\n",str,str1);
```

Che succede?

# string.h

**TABLE 8.1** Some String Library Functions from string.h

| Function | Purpose: Example | Parameters | Result Type |
|---|---|---|---|
| `strcpy` | Makes a copy of `source`, a string, in the character array accessed by `dest`: `strcpy(s1, "hello");` | `char *dest`<br>`const char *source` | `char *` h e l l o \0 ? ? ... |
| `strncpy` | Makes a copy of up to `n` characters from `source` in `dest`: `strncpy(s2, "inevitable", 5)` stores the first five characters of the source in `s1` and does NOT add a null character. | `char *dest`<br>`const char *source`<br>`size_t`† `n` | `char *` i n e v i ? ? ... |
| `strcat` | Appends `source` to the end of `dest`: `strcat(s1, "and more");` | `char *dest`<br>`const char *source` | `char *` h e l l o a n d  m o r e \0 |
| `strncat` | Appends up to `n` characters of `source` to the end of `dest`, adding the null character if necessary: `strncat(s1, "and more", 5);` | `char *dest`<br>`const char *source`<br>`size_t`† `n` | `char *` h e l l o a n d  m \0 ? |
| `strcmp` | Compares `s1` and `s2` alphabetically; returns a negative value if `s1` should precede `s2`, a zero if the strings are equal, and a positive value if `s2` should precede `s1` in an alphabetized list: `if (strcmp(name1, name2) == 0)...` | `const char *s1`<br>`const char *s2` | `int` |

`size_t` is an unsigned integer

Functions whose names are in color change a string value, but do not take a parameter indicating the size of the destination string or the size of the string to copy, and therefore do not provide the programmer with a means of preventing buffer overflow.

# string.h

**TABLE 8.1** Some String Library Functions from string.h

| Function | Purpose: Example | Parameters | Result Type |
|---|---|---|---|
| `strncmp` | Compares the first `n` characters of `s1` and `s2` returning positive, zero, and negative values as does `strcmp`: `if (strncmp(n1, n2, 12) == 0)` . . . | `const char *s1`<br>`const char *s2`<br>`size_t`† `n` | `int` |
| `strlen` | Returns the number of characters in `s`, not counting the terminating null: `strlen("What")` returns `4`. | `const char *s` | `size_t` |
| `strtok` | Breaks parameter string source into tokens by finding groups of characters separated by any of the delimiter characters in `delim`. First call must provide both `source` and `delim`. Subsequent calls using NULL as the source string find additional tokens in original source. Alters source by replacing first delimiter following a token by '\0'. When no more delimiters remain, returns rest of source. For example, if `s1` is `"Jan.12,.1842"`, `strtok(s1,".,")` returns `"Jan"`, then `strtok (NULL,".,")` returns `"12"` and `strtok(NULL,".,")` returns `"1842"`. The memory in the right column shows the altered `s1` after the three calls to `strtok`. Return values are pointers to substrings of `s1` rather than copies. | `const char *source`<br>`const char *delim` | `char *` J a n \0 1 2 \0 1 8 4 2 \0 |

`size_t` is an unsigned integer

Functions whose names are in color change a string value, but do not take a parameter indicating the size of the destination string or the size of the string to copy, and therefore do not provide the programmer with a means of preventing buffer overflow.